Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	26 February 2026
Team ID	699d5fc83074df695bbbb0c1
Project Name	AI Financial Advisor
Maximum Marks	4 Marks

Technical Architecture:

Architecture Diagram (3 Infrastructure Zones)

Zone	Color	What's inside
Local / Client	Green dashed	Streamlit UI → finance_analysis → Visualization → Utils → Session State → .env
Google Cloud	Blue dashed	Gemini 2.5 Flash LLM → 3 AI functions (advice, goal plan, chatbot)
Config / Admin	Gold dashed	API Key management → config.py → dependency manager

6-Step Data Flow numbered badges showing the full request lifecycle from user input → local compute → Gemini API → response parsing → visualisation.

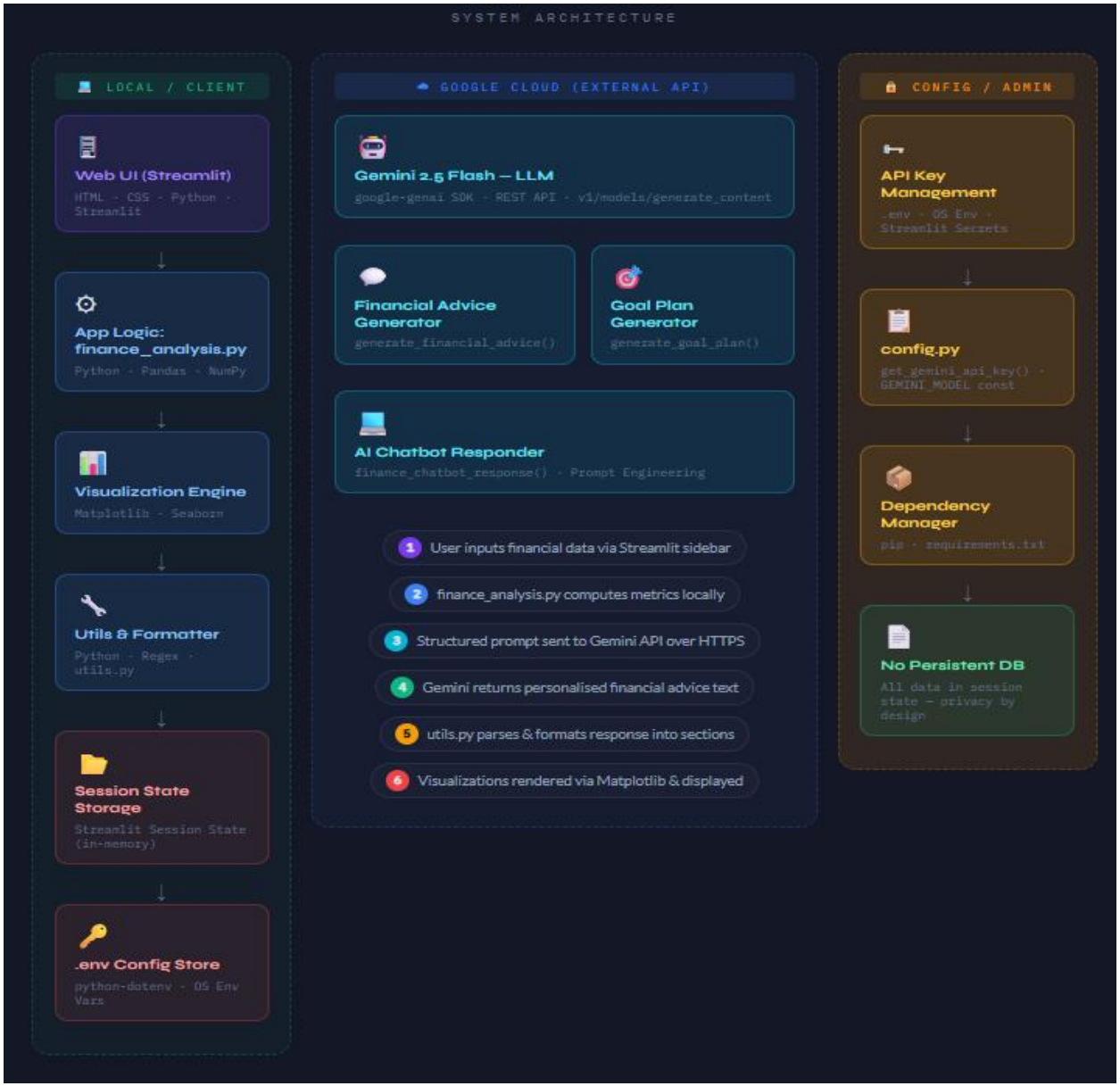
SYSTEM ARCHITECTURE
LOCAL / CLIENT
Web UI (Streamlit)
HTML · CSS · Python · Streamlit
App Logic: finance_analysis.py
Python · Pandas · NumPy
Visualization Engine
Matplotlib · Seaborn
Utils & Formatter
Python · Regex · utils.py
Session State Storage
Streamlit Session State (in-memory)
.env Config Store
python-dotenv · OS Env Vars
GOOGLE CLOUD (EXTERNAL API)
Gemini 2.5 Flash — LLM
google-genai SDK · REST API · v1/models/generate_content
Financial Advice Generator
generate_financial_advice()
Goal Plan Generator
generate_goal_plan()
AI Chatbot Responder
finance_chatbot_response() · Prompt Engineering
1 User inputs financial data via Streamlit sidebar
2 finance_analysis.py computes metrics locally
3 Structured prompt sent to Gemini API over HTTPS
4 Gemini returns personalised financial advice text
5 utils.py parses & formats response into sections
6 Visualizations rendered via Matplotlib & displayed
CONFIG / ADMIN
API Key Management
.env · OS Env · Streamlit Secrets
config.py
get_gemini_api_key() · GEMINI_MODEL const
Dependency Manager
pip · requirements.txt
No Persistent DB
All data in session state — privacy by design

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
01	**User Interface**	Streamlit-based web app served locally. Users enter financial data via sidebar forms, interact with tabs, chatbot and visualisations.	Streamlit, HTML/CSS, Python 3.10+
02	**Application Logic — Finance Analysis**	Computes savings, debt-to-income ratio, investment capacity, emergency fund gap and net worth from user inputs. All processing is local.	Python, Pandas, NumPy
03	**Application Logic — AI Advisor**	Constructs structured prompts and calls the Gemini API to generate personalised financial advice, goal plans and chatbot responses.	google-genai SDK Python Prompt Engineering
04	**Application Logic — Visualisation**	Generates pie charts (income allocation), bar charts (financial impact), gauge bars (debt health) and savings performance graphs.	Matplotlib Seaborn
05	**Session State / In-Memory Store**	Streamlit session_state preserves user data, analysis results, advice text, goal plan and chat history across reruns within a session.	Streamlit Session State Python dict
06	**Cloud Database / Storage**	No cloud database used. User data is never persisted to disk or cloud. Privacy-by-design — data lives only in session memory.	None (Privacy Design)
07	**File Storage**	Local filesystem stores the .env config file (API keys), styles.css and Python source modules. No user data is written to disk.	Local Filesystem python-dotenv
08	**External API — Gemini LLM**	Google Gemini 2.5 Flash model accessed via REST API. Used for generating financial advice, goal plans and chatbot responses.	Google Gemini 2.5 Flash HTTPS REST google-genai
09	**External API — Font CDN**	Google Fonts CDN loaded in the HTML interface for typography rendering (Playfair Display, DM Sans).	Google Fonts CDN HTTPS
10	**Machine Learning Model**	Gemini 2.5 Flash — a large language model (LLM) used for natural language generation of personalised financial advice, goal-oriented planning and conversational Q&A.	LLM (Transformer) Gemini 2.5 Flash NLG / NLU
11	**Infrastructure**	**Local:** Python 3.10+ on Windows/Mac/Linux, Streamlit dev server on port 8501. **Cloud:** Streamlit Community Cloud (optional deployment) or any cloud VM with Python runtime.	Local: Python 3.10+ Cloud: Streamlit Cloud Optional: Docker

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
01	**Open-Source Frameworks**	All core libraries are open-source. Streamlit (Apache 2.0), Matplotlib (PSF), Seaborn (BSD-3), Pandas (BSD-3), NumPy (BSD-3) and python-dotenv (BSD-3) are freely available and community maintained.	Streamlit (Apache 2.0) Pandas (BSD-3) NumPy (BSD-3) Matplotlib (PSF) Seaborn (BSD-3)
02	**Security Implementations**	API keys are never hardcoded — stored in .env and loaded via python-dotenv. All Gemini API calls use HTTPS/TLS encryption. No user financial data is persisted to disk or cloud, ensuring data privacy. Streamlit Secrets can be used for cloud deployments. Input validation prevents zero-income or empty-goal submissions.	HTTPS / TLS 1.3 python-dotenv Env Var Isolation No Data Persistence Input Validation
03	**Scalable Architecture**	The application follows a modular 3-tier architecture: Presentation Layer (Streamlit UI), Business Logic Layer (finance_analysis, ai_advisor, utils) and Service Layer (Gemini API). Each module is independently replaceable. The app can be containerised (Docker) and deployed on Kubernetes for horizontal scaling.	3-Tier Modular Docker (optional) Kubernetes (optional) Stateless Design
04	**Availability**	The local Streamlit server is single-instance but stateless — it can be restarted without data loss (session data is ephemeral by design). For cloud deployment on Streamlit Community Cloud or GCP, auto-restart and health checks ensure high availability. Gemini API has Google-backed 99.9% SLA.	Streamlit Cloud Auto-Restart Google API SLA 99.9% Stateless Sessions
05	**Performance**	Financial computations in finance_analysis.py run in under 5ms locally. Gemini API response time averages 2–5 seconds. Streamlit's session state caches analysis results to avoid recomputation on every rerun. st.spinner() provides visual feedback during API calls. matplotlib uses Agg backend (non-interactive) for fast server-side rendering.	Streamlit Session Cache Matplotlib Agg Backend st.spinner() UX Local CPU Compute